Verifying the Stability of Terraform Executions

Eloi Perdereau
eloi.perdereau@univ-orleans.fr
Univ. Orléans, INSA CVL, LIFO
France

Daniel Sokolowski
science@d.sokolowski.org
Independent Researcher
USA

Jolan Philippe
jolan.philippe1@univ-orleans.fr
Univ. Orléans, INSA CVL, LIFO
France

Hélène Coullon
helene.coullon@imt-atlantique.fr
IMT Atlantique, Inria, LS2N
France

Guido Salvaneschi
guido.salvaneschi@unisg.ch
University of St. Gallen
Switzerland

Abstract

With provisioning engines like Terraform, users declare their target infrastructure state, and the engine automatically plans and executes the required actions to reconcile the target and the actual state. However, ensuring the correctness and safety of such plans remains a challenge. For example, dependencies among resources may not be explicit, leading to potential security vulnerabilities, or deployment instability.

In this work, we introduce a generic property to verify the *stability* of Terraform changes, based on a mechanized semantics and integrated into a tool. Our tool augments the output of `terraform plan` by showing the action sequence and stability properties it implies. Evaluation experiments demonstrates the correctness of our semantics and the scalability of our tool on synthetic and real-world use cases.

ACM Reference Format:
Eloi Perdereau, Daniel Sokolowski, Jolan Philippe, Hélène Coullon, and Guido Salvaneschi. 2026. Verifying the Stability of Terraform Executions. In *Proceedings of CONFLANG'26*. ACM, New York, NY, USA, 2 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

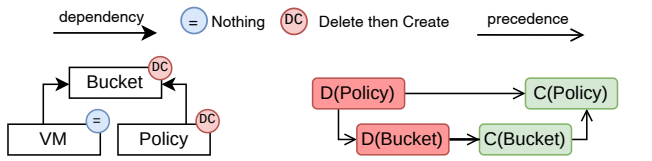
1 Introduction

Managing the infrastructure underlying software applications is a complex and, traditionally, manual task. Cloud providers, e.g., Google Cloud Platform, offer APIs to manage infrastructure resources, e.g., Virtual Machines (VMs) and networks, through create, read, update and delete (CRUD) actions. Infrastructure as Code (IaC) [6] provisioning engines such as Terraform [4] and Pulumi [3] offer a unified declarative interface to such APIs, abstracting over *change* operations: cloud providers maintain plugins that implement the concrete CRUD actions and users specify their *target state*, i.e., resource configuration and dependencies. The engine then plans and executes the actions required to reach that state. However, Terraform does not provide any information about the execution *schedule* of actions, though some action orders can result in instabilities or vulnerabilities during changes, an issue recognized as an intra-change safety concern [5].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CONFLANG'26, Munich, Germany
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

```
1  resource "google_storage_bucket" "Bucket" {
2    name = "storage-bucket" # <-- changed
3    lifecycle { create_before_destroy = false }
4  }
5  resource "google_compute_instance" "VM" {
6    boot_disk { ... }
7    depends_on = [ google_storage_bucket.Bucket ]
8  }
9  resource "google_storage_bucket_iam_member" "Policy" {
10   bucket = google_storage_bucket.Bucket.name
11 }
```

Listing 1: Subset of a Terraform manifest.



(a) Target state and plan, known before execution

(b) Action schedule, unknown before execution

Figure 1: Visualization of the graphs for Listing 1

2 Background

In Terraform, users write manifests that declare the configuration parameters and lifecycle behavior of resources to provision. Since resources may depend on each other, a manifest can be seen as a dependency graph between resources, i.e., the target state. In Listing 1, three resources are declared: a VM, a bucket (object storage) and a security policy. The VM and the policy depend on the bucket, specified at Lines 7 and 10, respectively. Terraform runs such manifests in two steps. (1) The target state is compared with the current state of the infrastructure, inferring a *plan*, that is, the operations to be applied on each resource: either (`N`)othing, (`C`)reate, (`D`)elete, (`U`)pdate or a replacement (`C-D` or `D-C`, i.e., create then delete or the reverse). (2) The operations are scheduled as individual `C`, `D` and `U` actions; *executed* in order; and the current state is updated. For example, changing the bucket name at Line 2 in Listing 1 results in (1) the planned operations depicted with the dependency graph in Figure 1a, and (2) the action schedule (a.k.a. the *action graph*) depicted in Figure 1b.

While Terraform allows users to inspect the plan, the schedule remains implicit and the problematic action orderings are only discovered during execution. For example, Figure 1b shows that the bucket is deleted while the VM is still running, and that the policy is created after the bucket, thus exposing the system to downtime and attacks, respectively. A possible fix for the latter is to change the value at Line 3 to `true` in order to create a new bucket before deleting the old one. In this case, however, two instances of the storage exist at the same time, which might induce unwanted costs. When solely relying on the plan, Terraform users have no visibility about the expected behavior of their infrastructure changes, the implied tradeoffs, e.g., cost vs. availability, and the potential instabilities induced by the execution schedule. Moreover, the specifications of Terraform remain informal and lack a precise semantics for reasoning about executions.

3 Contributions

To tackle this issue, we formalize the semantics of Terraform scheduling and execution, and we define a general stability property in Linear Temporal Logic [7] that verifies the degree of stability of a change. Intuitively, if the state of the system is always correct, the change is *stable*. If an incorrect state occurs and the fault is transient, i.e., the system recovers before the end of the execution, then the change is *eventually stable*. Finally, if an incorrect state persists after the end of the execution, the change is *perpetually unstable*. Incorrect states are characterized by a *faulty* predicate. We identified two such predicates: (1) a resource always has a unique instance (it is not *duplicated*), and (2) a resource never disappears when other resources depend on it (the dependency is not *dangling*). The semantics is implemented in Maude [2] and integrated into a tool that reads a Terraform plan and outputs an annotated plan produced by model-checking the change w.r.t. our properties. In Figure 1a for example, our tool would annotate the dependency between the VM and the bucket as *dangling*. Through synthetic cases and real Terraform manifests found in the TerraDS dataset [1], experimental results show the correctness of our semantics w.r.t. Terraform traces. Moreover, scalability experiments shows substantial variations according to the topology of the dependency graph, but also that instabilities are found rapidly.

In this presentation, we will explain our semantics of Terraform, the stability properties and the evaluation results, showing the ability of our tool to capture problematic executions in a preliminary phase. Moreover, the formal reasoning about Terraform executions allowed by our semantics opens the way to the design of static-analysis algorithms that would improve scalability. Finally, our work could be extended to account for other provisioning engines, such as Pulumi.

References

[1] Christoph Bühler, David Spielmann, Roland Meier, and Guido Salvaneschi. 2025. TerraDS: A Dataset for Terraform HCL Programs. In *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. 654–658. doi:10.1109/MSR66628.2025.00101

[2] Manuel Clavel, Francisco Durán, Steven Eker, Santiago Escobar, Patrick Lincoln, Narciso Martı-Oliet, José Meseguer, Rubén Rubio, and Carolyn Talcott. 2024. Maude manual (version 3.4). https://maude.lcc.uma.es/maude-manual/

[3] Pulumi Corporation. 2017. Pulumi. https://www.pulumi.com/.

[4] HashiCorp. 2014. Terraform. https://developer.hashicorp.com/terraform.

[5] Julien Lepiller, Ruzica Piskac, Martin Schäf, and Mark Santolucito. 2021. Analyzing Infrastructure as Code to Prevent Intra-update Sniping Vulnerabilities. In *Tools and Algorithms for the Construction and Analysis of Systems, TACAS 2021, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2021, Luxembourg City, Luxembourg, March 27 - April 1, 2021, Proceedings, Part II (Lecture Notes in Computer Science, Vol. 12652)*, Jan Friso Groote and Kim Guldstrand Larsen (Eds.). Springer, 105–123. doi:10.1007/978-3-030-72013-1_6

[6] Kief Morris. 2021. *Infrastructure as Code: Dynamic Systems for the Cloud Age* (second ed.). O'Reilly Media, Inc., Sebastopol, CA, USA.

[7] Amir Pnueli. 1977. The Temporal Logic of Programs. In *18th Annual Symposium on Foundations of Computer Science, Providence, Rhode Island, USA, 31 October - 1 November 1977*. IEEE Computer Society, 46–57. doi:10.1109/SFCS.1977.32